USN | | | | | | | | | | |

NMAM INSTITUTE OF TECHNOLOGY, NITTE

Off-Campus Centre of Nitte (Deemed to be University)

Fourth Semester B.Tech (CBCS) Degree Examinations

May 2025

CS3004-1 – DESIGN AND ANALYSIS OF ALGORITHMS

(For CS, IS , AM, CC, AD,CB)

Duration: 3 Hours | Max. Marks: 100

Note:

Part – A: Multiple Choice Questions: Answer all **Twenty** questions in the **OMR Sheet** provided. Each question carries equal marks.

Part – B: Descriptive Answer Questions: Answer **Five full** questions choosing **Two full** questions from **Unit – I** & **Unit – II each** and **One full** question from **Unit – III.**

PART - A: MULTIPLE CHOICE QUESTIONS

20 Marks

1. **How many sub arrays does the quick sort algorithm divide the entire array into?**
   A) One B) Two C) Three D) Four
2. **Binary Search works only on which type of arrays?**
   A) Unsorted arrays B) Sorted arrays C) Arrays with distinct elements D) Arrays of integers
3. **Linear search algorithm, also known as____**
   A) Binary search B) Simple search C) Sequential search D) Difficult search
4. **In asymptotic analysis, what does it mean if a function is said to be O(n)?**
   A) The time complexity increases exponentially as n increases
   B) The algorithm's runtime grows linearly with the input size
   C) The runtime of the algorithm is independent of the input size
   D) The algorithm's runtime grows logarithmically as n increases
5. **What will be the output of the following pseudocode if the driver is fun(5)? function fun(n) if n == 1 return 1 else return n + fun(n-1)**
   A) 120 B) 24 C) 6 D) 15
6. **Pseudocode is written using**
   A) Combination of Native & programming language
   B) High level Programming language only
   C) Natural language only
   D) Low level programming language
7. **Which of the following is NOT a characteristic of a good algorithm?**
   A) Clarity and simplicity B) Efficiency in terms of time and space C) Correctness D) The use of complex data structures
8. **What is the primary characteristic of the Divide and Conquer approach?**
   A) Solving problems by iteration
   B) Solving problems by dividing them into overlapping sub problems
   C) Dividing a problem into smaller independent subproblems, solving them recursively, and combining their results
   D) Dividing a problem into smaller problems and linearly solving it.
9. **Which technique is used to solve the 0/1 Knapsack problem efficiently?**
   A) Greedy algorithm B) Divide and conquer C) Dynamic programming D) Depth-first search

10. **What is the primary purpose of the Floyd-Warshall algorithm?**
    A) Finding the shortest path between two specific nodes
    B) Finding the shortest paths between all pairs of nodes in a graph
    C) Finding the minimum spanning tree of a graph
    D) Finding the longest path in a weighted graph

11. **In Horspool's algorithm, which table is used to store the shift values for the pattern matching process?**
    A) Shift Table
    B) Hash Table
    C) Match Table
    D) Character Table

12. **What makes Counting Sort a distribution-based algorithm?**
    A) It uses element frequencies to determine sorted positions.
    B) It divides the array into smaller subarrays.
    C) It recursively splits and merges data.
    D) It compares each pair of elements to sort the array.

13. **You are given an array: [3, 9, 2, 1, 4, 8]. After building a max-heap from this array, what will be the root element?**
    A) 1
    B) 9
    C) 2
    D) 8

14. **What operation is performed on an AVL tree when the balance factor of a node becomes greater than 1 or less than -1?**
    A) Rotation
    B) Deletion
    C) Insertion
    D) Search

15. **Binary Search is a classic example of which variation of Decrease and Conquer?**
    A) Decrease by a constant
    B) Decrease by a constant factor
    C) Decrease by one
    D) Divide and conquer

16. **In a max-heap, which of the following is true?**
    A) The parent node is smaller than its child nodes
    B) The parent node is greater than or equal to its child nodes
    C) The tree is a binary search tree
    D) The elements are stored in ascending order

17. **In the Job Assignment Problem, what does the objective function typically aim to minimize?**
    A) The time to complete all tasks
    B) The number of tasks assigned
    C) The total cost of assigning tasks to workers
    D) The total time worked by all workers

18. **In the subset-sum problem, when do we backtrack?**
    A) When the sum of the selected elements exceeds the target sum
    B) When the sum of the selected elements is equal to the target sum
    C) When all elements have been considered
    D) When a subset has been found

19. **Dijkstra's Algorithm is the prime example for _____**
    A) Greedy algorithm
    B) Branch and bound
    C) Back tracking
    D) Dynamic programming

20. **Which of the following algorithms is the best approach for solving Huffman codes?**
    A) exhaustive search
    B) greedy algorithm
    C) brute force algorithm
    D) divide and conquer algorithm

PART - B: DESCRIPTIVE ANSWER QUESTIONS

Unit – I

			Marks	BT*	CO*	PO*
1.	a)	Explain the Performance Analysis of an algorithm.	8	L*2	1	1
	b)	Write the pseudocode to sort the elements in an array using selection sort and analyze its efficiency. Apply selection sort to sort the following elements in an array. 5, 4, 2, 1, 6, 3.	8	L3	2	2
2.	a)	Write the general plan for analyzing the efficiency of non recursive algorithm. Write the pseudocode to find the maximal element in an array and analyze its efficiency.	8	L3	1	2

	b)	Write Divide And Conquer recursive Merge sort algorithm and derive the time complexity of this algorithm.	8	L3	2	2
3.	a)	Write the algorithm for brute force string matching. Show the comparisons the Brute force string matching makes for the patternP=acbcac in the text T=acbcabccababcaacbcac.	8	L3	1	2
	b)	Illustrate Quick sort algorithm. Sort the following list using Merge sort.38, 27, 43, 3, 9, 82, 10.	8	L3	2	2
		Unit – II				
4.	a)	Write an insertion sorting algorithm using decrease and conquer technique. Apply the algorithm on the following numbers and trace the steps - 23,1,10,5,2	8	L3	3	2
	b)	Construct AVL tree for the following input by showing each step of the insertion process and type of rotation used for transformation. 63, 9, 19, 27, 18, 108, 99, 81	8	L3	4	2
5.	a)	Identify the pattern "BARBER" in the given text "JIM_SAW_ME_IN_A_BARBERSHOP" using Horspool's string-matching algorithm. Show the construction of shift table for the same.	8	L3	3	2
	b)	Write an algorithm for sorting list of integers by comparison method that uses input enhancement design technique. Apply the algorithm on the following numbers and trace the steps 25,45,10,20,50,15.	8	L3	4	2
6.	a)	Define topological sorting. Apply DFS based method to topologically sort the vertices of the following graph.				

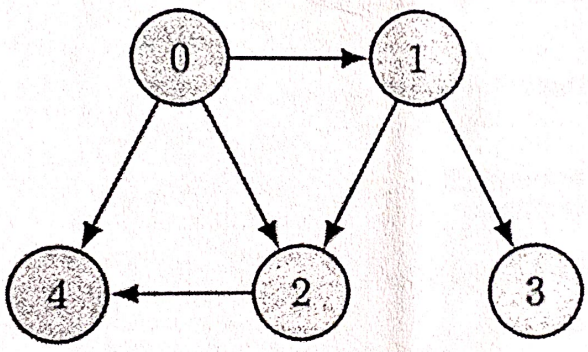


Fig. 6a

	b)	Write and Apply Floyd's all pairs shortest path algorithm for the graph shown in the following figure.	8	L3	3	2

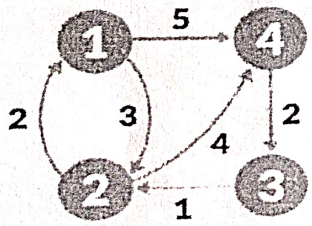


Fig. 6b

		Unit – III				
7.	a)	Solve the given instance of sum of subset problem S={5,10,12,13,15,18} and d=30. Construct a state space tree.	8	L3	4	2
	b)	Define minimum cost spanning tree. Write Prim's algorithm to find minimum cost spanning tree.	8	L3	5	2
			8	L2	5	1

3. a) Explain the following: (i) Class P (ii) Class NP (iii) NP Complete Problem (iv) NP Hard Problem. 8 L

b) Solve the job assignment problem using Branch and Bound technique for the following matrix.

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Fig. 8b

8 L

BT* Bloom's Taxonomy, L* Level; CO* Course Outcome; PO* Program Outcome.

NITTE
(Deemed to be University)

(Established under Sec. (3) of UGC Act, 1956)
Placed under Category 'A' by MHRD, GOI.
Accredited as 'A+' Grade by NAAC

Set 3

Scheme & Solution

Course Title :	**Design and analysis of algorithms**	**Course Code:**	**CS3004-1**
Prepared by:	**Dr. Deepa**	**Set No.:**	**2**
Faculty Signature:		**BOE Chairman Signature:**	

Note to QP Setter: Submit the Answer Key for MCQs in the separate template provided (if applicable).

Q.No.	Solution	Marks
1a	Explain the Performance Analysis of an algorithm.[2M for types+6M for key elements] There are two kinds of efficiency: time efficiency and space efficiency ***Time efficiency***, also called ***time complexity***, indicates how fast an algorithm in question runs. ***Space efficiency***, also called ***space complexity***, refers to the amount of memory units required by the algorithm in addition to the space needed for its input and output Key components of an algorithm analysis framework: **Measuring Input size** Defining the appropriate metric to represent the size of the input data (e.g., number of elements in an array). **Basic operations:** Identifying the fundamental operations performed by the algorithm, which are used to calculate the overall time complexity. **Time complexity analysis:** • **Asymptotic notation (Big O, Big Omega, Big Theta):** Used to describe the growth rate of an algorithm as the input size approaches infinity, focusing on the dominant term. • **Worst-case analysis:** Analyzing the maximum time an algorithm could take for a given input size. • **Average-case analysis:** Analyzing the expected running time for a typical input distribution. • **Best-case analysis:** Analyzing the minimum time an algorithm could take for a given input size (usually not very relevant for practical purposes).	8M

1b	Write the pseudocode to sort the elements in an array using selection sort and analyze its efficiency. Apply selection sort to sort the following elements in an array. 5, 4, 2, 1, 6, 3.[4M for pseudocode+2 M for analysis +2M for solving]
	ALGORITHM SelectionSort(A[0..n−1])
	//Sortsagivenarraybyselectionsort
	//Input:AnarrayA[0..n−1]oforderableelements
	//Output:ArrayA[0..n−1]sortedinnondecreasingorder
	fori←0ton−2do
	min←i
	forj←i+1ton−1do
	ifA[j]<A[min] min←j
	swapA[i]andA[min]
		8M

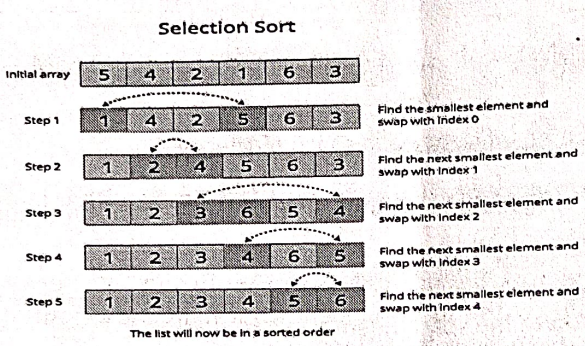
Time complexity

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=i}^{n-2} c = c \sum_{i=0}^{n-2} (n-i-1)$$

$$T(n) = c[n(n-1) - \frac{(n-2)(n-1)}{2} - (n-1)] = c(n-1)(n - \frac{(n-2)}{2} - 1)$$

$$T(n) = c(n-1)(\frac{n}{2})$$

$$T(n) \in \Theta(n^2)$$



2a Write the general plan for analyzing the efficiency of non recursive algorithm. Write the pseudocode to find the maximal element in an array and analyze its efficiency[4M +4M] 8M

General PlanforAnalyzingtheTimeEfficiencyofNonrecursiveAlgorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation. (As a rule, it is located in the inner most loop.)
3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case efficiencies have to be investigated separately.
4. Setup Sum Expressing the number of times the algorithm's basic operation is executed.4
5. Using Standard formulas and rules of sum manipulation, either find a closed form formula for the count or, at the very least, establish its order of growth.

ALGORITHM MaxElement(A[0..n−1])
//Determines the value of the largest element in a given array
//Input: An array A[0..n − 1]of real numbers
//Output: The value of the largest element in A
maxval ←A[0]
for i ←1ton−1do
if A[i]>maxval
maxval ←A[i]
return maxval

$$C(n) = \sum_{i=1}^{n-1} 1.$$

This is an easy sum to compute because it is nothing other than 1 repeated $n-1$ times. Thus,

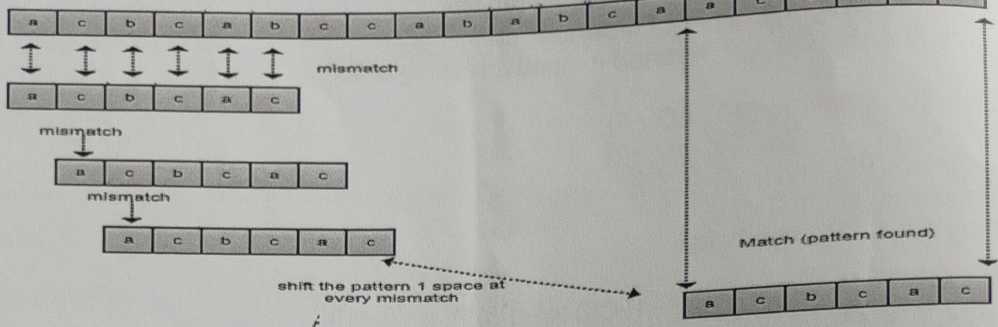
$$C(n) = \sum_{i=1}^{n-1} 1 = n-1 \in \Theta(n).$$

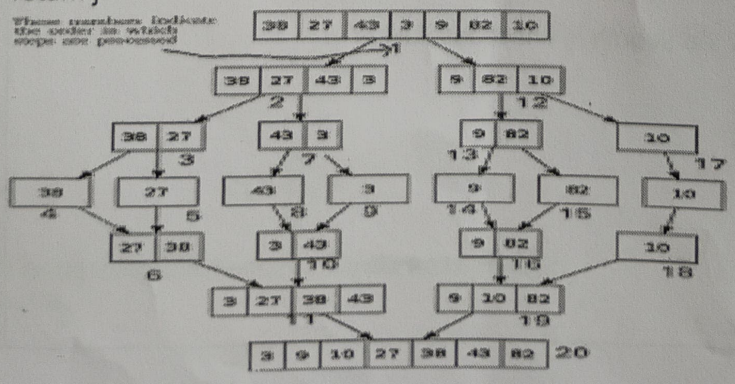
	Write Divide And Conquer recursive Merge sort algorithm and derive the time complexity of this algorithm.[6M +2M]	8M

b	ALGORITHM Mergesort(A[0..n−1])
	//Sorts array A[0..n − 1]by recursive mergesort
	//Input: An array A[0..n − 1]of orderable elements
	//Output: Array A[0..n − 1]sorted in nondecreasing order
	if n>1
	copy A[0.. n/2 −1]to B[0.. n/2 −1]
	copy A[n/2 ..n−1]to C[0.. n/2 −1]
	Mergesort(B[0.. n/2 −1])
	Mergesort(C[0.. n/2 −1])
	Merge(B, C, A) //see below
	
	ALGORITHM Merge(B[0..p −1],C[0..q −1],A[0..p +q −1])
	//Merges two sorted arrays into one sorted array
	//Input: Arrays B[0..p − 1]and C[0..q − 1]both sorted
	//Output: Sorted array A[0..p + q − 1]of the elements of B and C
	i ←0; j ←0; k←0
	while i<pand j<q do
	if B[i]≤ C[j]
	A[k]←B[i]; i ←i +1
	else A[k]←C[j]; j ←j +1
	k ←k+1
	if i =p
	copy C[j..q − 1]to A[k..p + q −1]
	else copy B[i..p − 1]to A[k..p + q −1
	
	Basic -
	$C_{worst}(n) = \begin{cases} 0 & \text{if } n = 1 \\ 2C_{worst}(\frac{n}{2}) + n-1 & \text{if } n > 1 \end{cases}$
	According to master's Theorem.
	$T(n) = aT(\frac{n}{b}) + f(n)$
	By comparison,
	$C(n) = 2C(\frac{n}{2}) + n-1$
	$a = 2, \quad b = 2, \quad f(n) = n-1$
	$f(n) \in \Theta(n) \Rightarrow d = 1$
	$a = 2 \quad b^d = 2^1 = 2$
	$a = b^d$
	$\rightarrow T(n) \in \Theta(n^d \log n)$
	$\in \Theta(n^1 \log n)$
	$T(n) \in \Theta(n \log n)$	

3a	Write the algorithm for brute force string matching. Show the comparisons the Brute force string matching makes for the pattern P=acbcac in the text T=acbcabccababcaacbcac.[4M +4M]	8M

	ALGORITHM BruteForceStringMatch(T[0..n −1],P[0..m −1])
	//Implements brute-force string matching
	//Input: An array T[0..n − 1]of n characters representing a text and
	//
	an array P[0..m −1]of m characters representing a pattern
	//Output: The index of the first character in the text that starts a
	//
	matching substring or −1if the search is unsuccessful
	for i ←0ton−mdo
	j ←0
	while j<mandP[j]=T[i +j]do
	j ←j+1
	if j =m return i
	return −1	



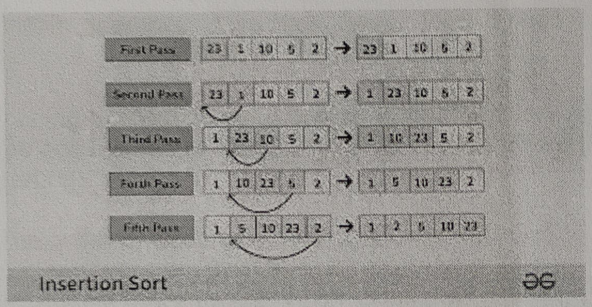
3b	Illustrate Quick sort algorithm.
	Sort the following list using Merge sort. 38, 27, 43, 3, 9, 82, 10.[4M +4M]
	ALGORITHM Quicksort(A[l..r])
	//Sorts a subarray by quicksort
	//Input: Subarray of array A[0..n − 1], defined by its left and right
	//
	indices l and r
	//Output: Subarray A[l..r] sorted in nondecreasing order
	if l<r
	s ←Partition(A[l..r]) //s is a split position
	Quicksort(A[l..s − 1])
	Quicksort(A[s + 1..r])
	
	ALGORITHM HoarePartition(A[l..r])
	as a pivot
	//Partitions a subarray by Hoare's algorithm, using the first element
	//Input: Subarray of array A[0..n − 1], defined by its left and right
	//indices l and r(l<r)
	//Output: Partition of A[l..r], with the split position returned as
	/ this function's value
	p ←A[l]
	i ←l; j ←r+1
	repeat
	repeat i ←i +1until A[i]≥p
	repeat j ←j −1until A[j]≤p
	swap(A[i],A[j])
	until i ≥ j
	swap(A[i],A[j]) //undo last swap when i ≥ j
	swap(A[l],A[j])
	return j
	 
		8M

a

Write an insertion sorting algorithm using decrease and conquer technique. Apply the algorithm on the following numbers and trace the steps - 23,1,10,5,2 [4M +4M]

8M

```
ALGORITHM InsertionSort(A[0..n −1])
 //Sorts a given array by insertion sort
 //Input: An array A[0..n − 1]of n orderable elements
 //Output: Array A[0..n − 1]sorted in nondecreasing order
 for i ←1ton−1do
 v ←A[i]
 j ←i−1
 while j ≥0 and A[j]>vdo
 A[j +1]←A[j]
 j ←j−1
 A[j +1]←v
```



Discuss before valuation.

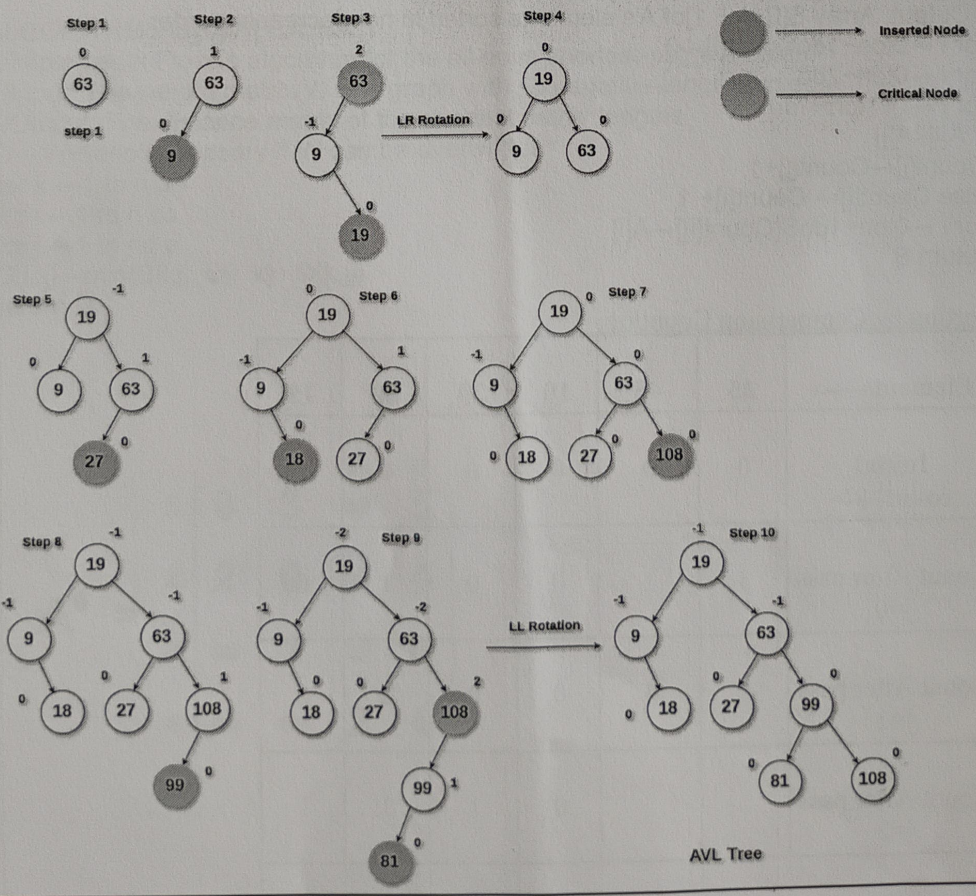
-23,1,10,5,2 → -23,1,10,5,2
-23,1,10,5,2 → -23,1,10,5,2
-23,1,10,5,2 → -23,1,10,5,2
-23,1,10,5,2 → -23,1,5,10,2
-23,1,5,10,2 → -23,1,2,5,10

(If not are assumed as -23,1,10,5,2)

4b

Construct AVL tree for the following input by showing each step of the insertion process and type of rotation used for transformation.
63, 9, 19, 27, 18, 108, 99, 81

8M



5a

Identify the pattern "BARBER" in the given text "JIM_SAW_ME_IN_A_BARBERSHOP" using Horspool's string-matching algorithm. Show the construction of shift table for the same.[2M for shift table+ 5M]

8M

character c	A	B	C	D	E	F	...	R	...	Z	_
shift $t(c)$	4	2	6	6	1	6	6	3	6	6	6

The actual search in a particular text proceeds as follows:

```
J I M _ S A W _ M E _ I N _ A _ B A R B E R S H O P
B A R B E R                 B A R B E R
        B A R B E R             B A R B E R
          B A R B E R                   B A R B E R
```

5b

Write an algorithm for sorting list of integers by comparison method that uses input enhancement design technique. Apply the algorithm on the following numbers and trace the steps 25,45,10,20,50,15.

8M

```
ALGORITHM ComparisonCountingSort(A[0..n−1])
//Sorts an array by comparison counting
//Input: An array A[0..n − 1]of orderable elements
//Output: Array S[0..n − 1]of A's elements sorted in nondecreasing order
for i ←0ton−1doCount[i]←0
for i ←0ton−2do
for j ←i +1ton−1do
if A[i]<A[j]
Count[j]←Count[j]+1
else Count[i]← Count[i]+ 1
for i ←0ton−1doS[Count[i]]←A[i]
return S
```

Sorting by Comparision Counting:

Elements---->	**25**	**45**	**10**	**20**	**50**	**15**
Initial count---->	**0**	**0**	**0**	**0**	**0**	**0**
Count After pass j=0	3	1	0	0	1	0
Count After pass j=1		4	0	0	2	0
Count After pass j=2			0	1	3	1
Count After pass j=3				2	4	1

Count After pass j=4					5	1
Final state	**3**	**4**	**0**	**2**	**5**	**1**

OUTPUT:

Sorted Array	10	15	20	25	45	50

6a Define topological sorting. Apply DFS based method to topologically sort the vertices of the following graph.[2M +6M] 8M

Listing vertices in such an order that for every edge in the graph, the vertex where the edge starts is listed before the vertex where the edge ends is called topological sorting

The order of visiting 0 , 1, 3, 2, 4
Order of popping 3, 4, 2, 1, 0
Toplogical order 0, 1, 2, 4, 3

6b Write and Apply Floyd's all pairs shortest path algorithm for the graph shown in the following figure[4M +4M] 8M

ALGORITHM Floyd(W[1..n,1..n])
//Implements Floyd's algorithm for the all-pairs shortest-paths problem
//Input: The weight matrix W of a graph with no negative-length cycle
//Output: The distance matrix of the shortest paths' lengths
D←W //isnotnecessary if W can be overwritten
for k ←1 to n do
for i ←1 to n do
for j ←1 to nd o
D[i, j]←min{D[i,j], D[i, k]+ D[k,j]}
return D

$$A^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{array}$$

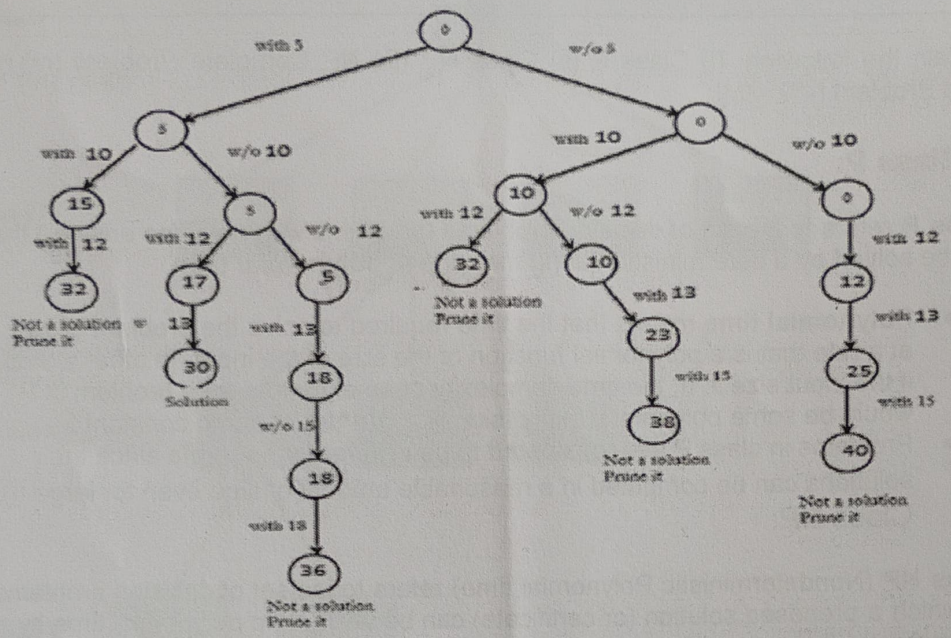
$$A^1 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{array} \longrightarrow \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & 9 & 4 \\ \infty & 1 & 0 & 8 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{array}$$

$$A^2 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & & \\ 2 & 0 & 9 & 4 \\ & 1 & 0 & \\ & \infty & & 0 \end{bmatrix} \end{array} \longrightarrow \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 1 & 0 & 5 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{array}$$

$$A^3 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & & \infty & \\ & 0 & 9 & \\ \infty & 1 & 0 & 8 \\ & & 2 & 0 \end{bmatrix} \end{array} \longrightarrow \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{array}$$

$$A^4 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & & & 5 \\ & 0 & & 4 \\ & & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{array} \longrightarrow \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & 6 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{array}$$

Solve the given instance of sum of subset problem S={5,10,12,13,15,18} and d=30. Construct a state space tree. 8M



7b Define minimum cost spanning tree. Write Prim's algorithm to find minimum cost spanning tree. [2M +6M] 8M

A spanning tree of an undirected connected graph is itsconnected acyclic subgraph (i.e., a tree) that contains all the vertices of the graph. If such a graph has weights assigned to its edges, a minimum spanning tree is its spanning tree of the smallest weight, where the weight of a tree is defined as the sum of the weights on all its edges.

ALGORITHM Prim(G)

//Prim's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph G = V,E

//Output: ET, the set of edges composing a minimum spanning tree of G

VT ←{v0} //the set of tree vertices can be initialized with any vertex

ET ←∅

for i ←1to|V|−1do

f ind a minimum-weight edge e∗ =(v∗,u∗) among all the edges (v, u)

such that v is in VT and u is in V −VT

VT ←VT ∪{u∗}

ET ←ET ∪{e∗}

return ET

8a	Explain the following: (i) Class P (ii) Class NP (iii) NP Complete Problem (iv) NP Hard Problem.[4*2=8M]	8M

(i) Class P:

Class P refers to the set of decision problems (problems with a yes/no answer) that can be solved by a deterministic Turing machine in **polynomial time**.

- **Polynomial time** means that the time required to solve the problem grows at a rate that is a polynomial function of the size of the input. In other words, if the input size is **n**, the time complexity of an algorithm for a problem in P would be some polynomial expression like **O(n^k)** for some constant **k**.
- Problems in class P are considered to be **efficiently solvable** since their solutions can be computed in a reasonable amount of time even for large ii)

Class NP:

Class NP (Nondeterministic Polynomial time) refers to the set of decision problems for which a proposed solution (or certificate) can be **verified** in polynomial time by a deterministic Turing machine.

- A problem is in NP if, given a candidate solution, it can be checked quickly (in polynomial time) whether it is correct or not. However, it may not be known whether an optimal solution can be found in polynomial time.
- **NP does not necessarily imply that the problem can be solved in polynomial time**. It only suggests that solutions can be **verified** in polynomial time, even though finding the solution itself might take longer.

(iii) NP-Complete Problem:

An **NP-Complete** problem is a problem in NP that is as hard as any problem in NP. In other words, it is a problem in NP that **every other problem in NP can be reduced** to in polynomial time. If you can solve an NP-Complete problem in polynomial time, you can solve all problems in NP in polynomial time.

- NP-Complete problems are considered the **most difficult problems** in NP. If any NP-Complete problem has a polynomial-time solution, it would imply that **P = NP**, meaning all NP problems can be solved in polynomial time.
- These problems are typically used to represent the difficulty of finding a polynomial-time solution for problems in NP.

(iv) NP-Hard Problem:

An **NP-Hard** problem is a problem that is at least as hard as the hardest problems in NP. In other words, any problem in NP can be **reduced to** an NP-Hard problem in polynomial time.

- **NP-Hard does not necessarily mean that the problem is in NP**. It means the problem may not even be a decision problem (i.e., it may not have a yes/no answer) or it may not have a polynomial-time solution, but it is still hard in the sense that it is at least as hard as the hardest problems in NP.
- **NP-Hard problems** may not have the property that solutions can be verified in polynomial time (unlike NP problems), but they represent the upper-bound difficulty for problems.

Solve the given traveling salesperson problem using Branch and Bound technique.

8M